User Stories and Use Cases

Project: Real-Time IoT Data Pipeline **Team Leader:** Rana Shaaban Abdelqader Mohamed
Section: 3. Requirements Gathering

1. Purpose

This document describes how users and the system interact, first as short user stories and then as detailed use cases. Every story and use case traces back to a stakeholder need in the Stakeholder Analysis and to a requirement in the Functional Requirements document.

2. Actors

Actor	Description
Monitoring Operator	End user who watches the dashboard and reacts to alerts
Lecturer / Evaluator	Reviews the working system, the documentation, and the final demo
Data Engineer (team member)	Runs, configures, and maintains the pipeline components
Sensor Simulator	Automated actor that produces temperature and humidity readings
Snowflake	External system that stores the processed data

3. User Stories

Priority uses MoSCoW: **M** = Must have, **S** = Should have, **C** = Could have.

ID	As a...	I want to...	So that...	Priority
US-01	Data Engineer	have a simulator generate temperature and humidity readings every 5 seconds	the pipeline always has a continuous stream of data to process	M
US-02	Data Engineer	send each reading to a Flask API that publishes it to Kafka	readings are collected reliably through a message queue	M
US-03	Data Engineer	have invalid or incomplete readings rejected at the API	bad data does not enter the pipeline	S
US-04	Data Engineer	run a batch ETL job that cleans, flags anomalies, and computes averages	the data is ready for analysis	M
US-05	Data Engineer	load the cleaned data into Snowflake	the data can be stored and analyzed later	M
US-06	Monitoring Operator	receive an alert when temperature or humidity crosses a threshold	I notice abnormal values immediately instead of late	M
US-07	Monitoring Operator	see a live chart of temperature and humidity per sensor	I can follow sensor trends at a glance	M
US-08	Monitoring Operator	see the number of anomalies over time	I can judge how often sensors behave abnormally	M
US-09	Monitoring Operator	browse the alert history	I can review past incidents and when they happened	M
US-10	Monitoring Operator	filter the dashboard by sensor ID or location	I can focus on one sensor or area	S
US-11	Data Engineer	change threshold values without editing the code	I can tune alert rules quickly	S

ID	As a...	I want to...	So that...	Priority
US-12	Lecturer	see the full documentation and code in a GitHub repository	I can evaluate the project against the course requirements	M
US-13	Lecturer	watch a live end-to-end demo	I can confirm that both the batch and streaming paths work	M
US-14	Data Engineer	see logs when a component fails	I can find and fix problems quickly	C

Acceptance Criteria (key stories)

Story	Acceptance Criteria
US-01	A new reading with sensor ID, timestamp, temperature, humidity, and location is produced every 5 seconds, matching the agreed schema
US-02	Every valid reading sent to the API appears in the Kafka topic with no loss during a normal run
US-05	At least 99% of generated readings are present in Snowflake after the ETL run
US-06	An alert is raised within 5 seconds of a threshold breach and shows sensor ID, value, threshold, and time
US-07	The live chart updates automatically without a manual page reload
US-09	Every alert raised by the streaming path is listed with its timestamp
US-12	The repository contains all weekly deliverables and all project documents

4. Use Case Diagram (Summary)

Actor	Use Cases
Sensor Simulator	UC-01 Generate Sensor Readings
Data Engineer	UC-02 Ingest Readings, UC-03 Run Batch ETL, UC-04 Process Stream and Raise Alert (configures rules)
Monitoring Operator	UC-05 View Live Dashboard, UC-06 View Alert History
Lecturer / Evaluator	UC-05 View Live Dashboard, UC-06 View Alert History

5. Use Case Specifications

UC-01: Generate Sensor Readings

Field	Description
Actor	Sensor Simulator
Goal	Produce a continuous stream of temperature and humidity readings
Precondition	The simulator is started and the schema is defined
Trigger	Simulator starts, then repeats every 5 seconds
Main Flow	1. The simulator creates a reading with sensor ID, timestamp, temperature, humidity, and location. 2. The simulator sends the reading to the Flask API. 3. The simulator waits 5 seconds and repeats.
Alternate Flow	A1. The API is unreachable: the simulator logs the error and retries on the next cycle.
Postcondition	Readings are delivered to the ingestion API

UC-02: Ingest Readings

Field	Description
Actor	Data Engineer (runs the component); Sensor Simulator (sends data)
Goal	Receive readings and publish them to Kafka
Precondition	Flask API and Kafka broker are running
Main Flow	1. The API receives a reading. 2. The API validates the fields against the schema. 3. The Kafka producer publishes the reading to the Kafka topic. 4. The API returns a success response.
Alternate Flow	A1. Validation fails: the API rejects the reading and returns an error. A2. Kafka is unavailable: the API returns an error and the failure is logged.
Postcondition	The valid reading is stored in the Kafka topic and available to both consumers

UC-03: Run Batch ETL

Field	Description
Actor	Data Engineer; Snowflake (external system)
Goal	Turn raw readings into clean, enriched data in the warehouse
Precondition	The batch Kafka consumer has written batched files
Main Flow	1. The batch consumer reads from the Kafka topic and writes batched files. 2. The ETL job loads a batched file. 3. The job cleans the data (missing values, duplicates, wrong types). 4. The job flags anomalies and computes averages. 5. The job loads the result into Snowflake.

Field	Description
Alternate Flow	A1. A record fails cleaning: it is excluded and logged. A2. The Snowflake load fails: the job logs the error and the file is kept so the load can be retried.
Postcondition	Cleaned and enriched data is stored in Snowflake

UC-04: Process Stream and Raise Alert

Field	Description
Actor	Data Engineer (configures rules); Monitoring Operator (receives alerts)
Goal	Detect threshold breaches in real time
Precondition	The streaming consumer is running and threshold rules are configured
Main Flow	1. The streaming consumer reads a reading from the Kafka topic. 2. The consumer compares temperature and humidity with the threshold rules. 3. If a threshold is breached, the consumer raises an alert with sensor ID, value, threshold, and time. 4. The alert is recorded and made available to the dashboard.
Alternate Flow	A1. No threshold is breached: no alert is raised and processing continues. A2. The reading is malformed: it is skipped and logged.
Postcondition	Every threshold breach produces one alert within 5 seconds

UC-05: View Live Dashboard

Field	Description
Actor	Monitoring Operator; Lecturer / Evaluator

Field	Description
Goal	See current sensor trends, anomaly counts, and alerts
Precondition	The dashboard is running and data exists in Snowflake and in the alert stream
Main Flow	1. The user opens the dashboard. 2. The dashboard loads processed data from Snowflake. 3. The dashboard shows temperature and humidity trends and anomaly counts. 4. The dashboard shows new alerts from the alert stream as they arrive. 5. The user optionally filters by sensor ID or location.
Alternate Flow	A1. Snowflake is unreachable: the dashboard shows a clear message and keeps showing the alert stream. A2. No data yet: the dashboard shows an empty-state message.
Postcondition	The user sees up-to-date metrics and alerts

UC-06: View Alert History

Field	Description
Actor	Monitoring Operator; Lecturer / Evaluator
Goal	Review past alerts
Precondition	At least one alert has been raised and recorded
Main Flow	1. The user opens the alert history section of the dashboard. 2. The dashboard lists alerts with timestamp, sensor ID, metric, value, and threshold. 3. The user optionally filters by sensor or time range.
Alternate Flow	A1. No alerts exist: the dashboard shows an empty-state message.
Postcondition	The user has reviewed past incidents

6. Traceability

Use Case	Related User Stories	Related Objective (Proposal 3.1)
UC-01	US-01	Simulate IoT sensor data every 5 seconds
UC-02	US-02, US-03	Ingest data through a Flask API and Kafka producer
UC-03	US-04, US-05	Build a batch ETL pipeline into Snowflake
UC-04	US-06, US-11	Build a streaming pipeline with real-time alerts
UC-05	US-07, US-08, US-10, US-13	Build a live dashboard
UC-06	US-09	Build a live dashboard (alert history)